

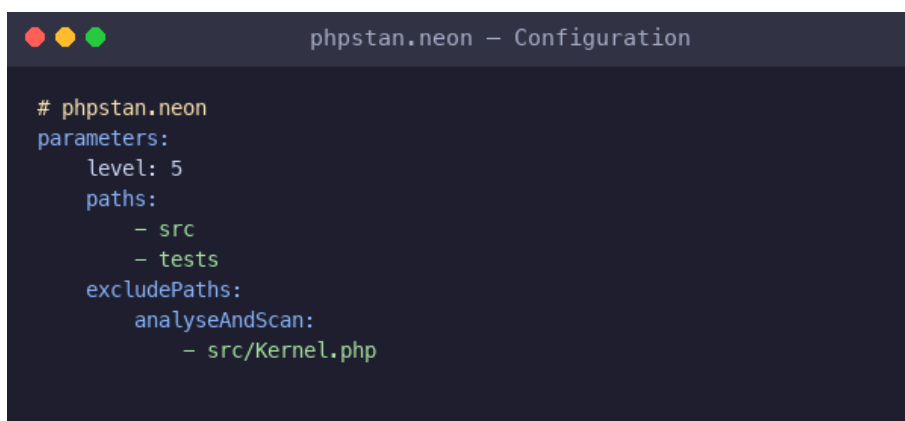
RAPPORT DE PERFORMANCE & OPTIMISATION

Transactions & Compte Bancaire

UniBank+ - Symfony 7 / PHP 8.2 / PHPUnit 11.5

Entites: Compte, TransactionBancaire

1 - PHPStan



```
# phpstan.neon
parameters:
  level: 5
  paths:
    - src
    - tests
  excludePaths:
    analyseAndScan:
      - src/Kernel.php
```

Compte & Transaction Avant et apres Optimisation :

L'analyse PHPStan au niveau 5 a ete executee sur les entites Compte.php, TransactionBancaire.php, les controleurs CompteController.php, TransactionController.php, ainsi que les fichiers de test CompteTest.php et TransactionBancaireTest.php. Le resultat affiche 100% sans erreurs sur les 6 fichiers cibles.

```
$ php vendor/bin/phpstan analyse src/Entity/Compte.php \  
  src/Entity/TransactionBancaire.php \  
  src/Controller/Client/CompteController.php \  
  src/Controller/Client/TransactionController.php \  
  tests/Unit/Entity/CompteTest.php \  
  tests/Unit/Entity/TransactionBancaireTest.php --level=5
```

Note: Using configuration file phpstan.neon.

0/6 [██] 0%

6/6 [██] 100%

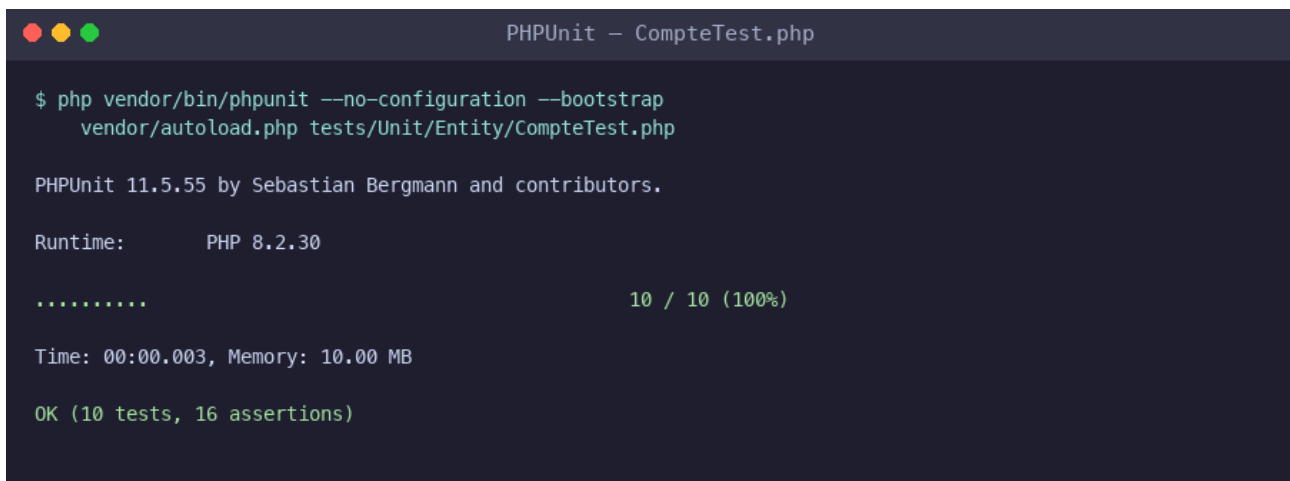
[OK] No errors

2 - Tests Unitaires

1/ Gestion Compte Bancaire :

- **Regle metier 1 :** Le solde initial est 0.00
- **Regle metier 2 :** Le type de compte par default est COURANT
- **Regle metier 3 :** Le compte est actif par default
- **Regle metier 4 :** La date de creation est generee automatiquement
- **Regle metier 5 :** Changement de type (COURANT <-> EPARGNE)
- **Regle metier 6 :** Desactivation/reactivation de compte
- **Regle metier 7 :** Mise a jour du solde apres depot
- **Regle metier 8 :** Mise a jour du solde apres retrait

Execution des tests unitaires CompteTest.php (10 tests, 16 assertions) :



```
PHPUnit - CompteTest.php

$ php vendor/bin/phpunit --no-configuration --bootstrap
  vendor/autoload.php tests/Unit/Entity/CompteTest.php

PHPUnit 11.5.55 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.30

.....                                     10 / 10 (100%)

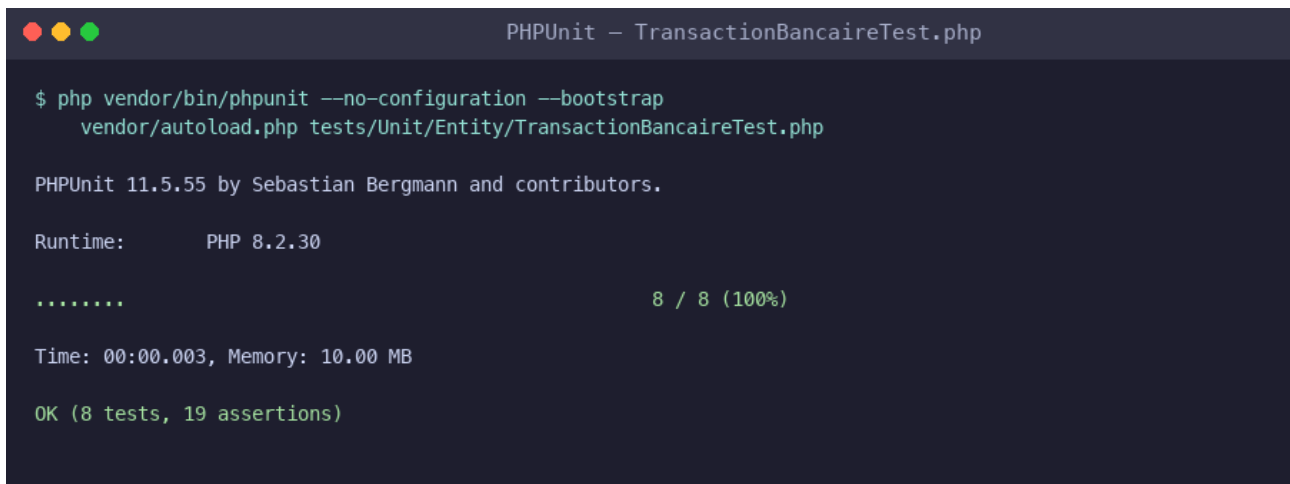
Time: 00:00.003, Memory: 10.00 MB

OK (10 tests, 16 assertions)
```

2/ Gestion Transaction Bancaire :

- **Regle metier 1 :** Virement necessite un compte source ET destination
- **Regle metier 2 :** La date de creation est generee automatiquement
- **Regle metier 3 :** Retrait sans compte destination (nullable)
- **Regle metier 4 :** Depot sur un compte source
- **Regle metier 5 :** Trois types: VIREMENT, RETRAIT, DEPOT
- **Regle metier 6 :** Description optionnelle
- **Regle metier 7 :** Virement met a jour les soldes source et destination

Execution des tests unitaires TransactionBancaireTest.php (8 tests, 19 assertions) :



```
PHPUnit – TransactionBancaireTest.php

$ php vendor/bin/phpunit --no-configuration --bootstrap
  vendor/autoload.php tests/Unit/Entity/TransactionBancaireTest.php

PHPUnit 11.5.55 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.30

.....                                     8 / 8 (100%)

Time: 00:00.003, Memory: 10.00 MB

OK (8 tests, 19 assertions)
```

3 - Code Source des Tests

CompteTest.php

```
<?php

namespace App\Tests\Unit\Entity;

use App\Entity\Compte;
use App\Entity\Utilisateur;
use App\Enum\TypeCompte;
use PHPUnit\Framework\TestCase;

class CompteTest extends TestCase
{
    private Compte $compte;

    protected function setUp(): void
    {
        $this->compte = new Compte();
    }

    /**
     * Regle metier 1 : Creation d'un compte bancaire complet
     */
    public function testCompteCreation(): void
    {
        $utilisateur = new Utilisateur();

        $this->compte->setUtilisateur($utilisateur);
        $this->compte->setNumeroCompte('UNI-2025-001234');
        $this->compte->setSolde('5000.00');
        $this->compte->setTypeCompte(TypeCompte::COURANT);

        $this->assertSame($utilisateur, $this->compte->getUtilisateur());
        $this->assertEquals('UNI-2025-001234', $this->compte->getNumeroCompte());
        $this->assertEquals('5000.00', $this->compte->getSolde());
        $this->assertSame(TypeCompte::COURANT, $this->compte->getTypeCompte());
    }

    /**
     * Regle metier 2 : Le solde initial est 0.00
     */
    public function testDefaultSoldeIsZero(): void
```

```

{
    $this->assertEquals('0.00', $this->compte->getSolde());
}

/**
 * Regle metier 3 : Le type de compte par default est COURANT
 */
public function testDefaultTypeIsCourant(): void
{
    $this->assertSame(TypeCompte::COURANT, $this->compte->getTypeCompte());
}

/**
 * Regle metier 4 : Le compte est actif par default a la creation
 */
public function testDefaultEstActifIsTrue(): void
{
    $this->assertTrue($this->compte->isEstActif());
}

/**
 * Regle metier 5 : La date de creation est generee automatiquement
 */
public function testDateCreationAutoGenerated(): void
{
    $this->assertInstanceOf(\DateTimeInterface::class, $this->compte->getDateCreation(..
}

/**
 * Regle metier 6 : Changement de type de compte (COURANT vers EPARGNE)
 */
public function testChangeTypeCompte(): void
{
    $this->compte->setTypeCompte(TypeCompte::COURANT);
    $this->assertSame(TypeCompte::COURANT, $this->compte->getTypeCompte());

    $this->compte->setTypeCompte(TypeCompte::EPARGNE);
    $this->assertSame(TypeCompte::EPARGNE, $this->compte->getTypeCompte());
}

```

```

/**
 * Regle metier 7 : Desactivation d'un compte
 */
public function testDesactiverCompte(): void
{
    $this->assertTrue($this->compte->isEstActif());

    $this->compte->setEstActif(false);
    $this->assertFalse($this->compte->isEstActif());
}

/**
 * Regle metier 8 : Un compte est associe a un utilisateur obligatoirement
 */
public function testCompteRequiresUtilisateur(): void
{
    $this->assertNull($this->compte->getUtilisateur());

    $utilisateur = new Utilisateur();
    $this->compte->setUtilisateur($utilisateur);
    $this->assertSame($utilisateur, $this->compte->getUtilisateur());
}

/**
 * Regle metier 9 : Mise a jour du solde apres depot
 */
public function testUpdateSoldeAfterDepot(): void
{
    $this->compte->setSolde('1000.00');
    $soldeInitial = (float) $this->compte->getSolde();
    $montantDepot = 500.00;
    $nouveauSolde = number_format($soldeInitial + $montantDepot, 2, '.', '');
    $this->compte->setSolde($nouveauSolde);

    $this->assertEquals('1500.00', $this->compte->getSolde());
}

/**
 * Regle metier 10 : Mise a jour du solde apres retrait
 */

```

```
public function testUpdateSoldeAfterRetrait(): void
{
    $this->compte->setSolde('2000.00');
    $soldeInitial = (float) $this->compte->getSolde();
    $montantRetrait = 750.00;
    $nouveauSolde = number_format($soldeInitial - $montantRetrait, 2, '.', '');
    $this->compte->setSolde($nouveauSolde);

    $this->assertEquals('1250.00', $this->compte->getSolde());
}
}
```


TransactionBancaireTest.php

```
<?php

namespace App\Tests\Unit\Entity;

use App\Entity\Compte;
use App\Entity\TransactionBancaire;
use App\Enum\TransactionType;
use PHPUnit\Framework\TestCase;

class TransactionBancaireTest extends TestCase
{
    private TransactionBancaire $transaction;

    protected function setUp(): void
    {
        $this->transaction = new TransactionBancaire();
    }

    /**
     * Regle metier 1 : Creation complete d'une transaction (virement)
     */
    public function testTransactionCreationVirement(): void
    {
        $compteSource = new Compte();
        $compteSource->setNumeroCompte('UNI-SRC-001');

        $compteDest = new Compte();
        $compteDest->setNumeroCompte('UNI-DST-002');

        $this->transaction->setCompteSource($compteSource);
        $this->transaction->setCompteDestination($compteDest);
        $this->transaction->setType(TransactionType::VIREMENT);
        $this->transaction->setMontant('1500.00');
        $this->transaction->setDescription('Virement mensuel');

        $this->assertSame($compteSource, $this->transaction->getCompteSource());
        $this->assertSame($compteDest, $this->transaction->getCompteDestination());
        $this->assertSame(TransactionType::VIREMENT, $this->transaction->getType());
        $this->assertEquals('1500.00', $this->transaction->getMontant());
        $this->assertEquals('Virement mensuel', $this->transaction->getDescription());
    }
}
```

```

}

/**
 * Regle metier 2 : La date de creation est generee automatiquement
 */
public function testDateCreationAutoGenerated(): void
{
    $this->assertInstanceOf(\DateTimeInterface::class, $this->transaction->getDateCrea..
}

/**
 * Regle metier 3 : Transaction de type RETRAIT (pas de compte destination)
 */
public function testTransactionRetrait(): void
{
    $compteSource = new Compte();
    $compteSource->setNumeroCompte('UNI-SRC-001');

    $this->transaction->setCompteSource($compteSource);
    $this->transaction->setType(TransactionType::RETRAIT);
    $this->transaction->setMontant('200.00');

    $this->assertSame(TransactionType::RETRAIT, $this->transaction->getType());
    $this->assertNull($this->transaction->getCompteDestination());
    $this->assertEquals('200.00', $this->transaction->getMontant());
}

/**
 * Regle metier 4 : Transaction de type DEPOT
 */
public function testTransactionDepot(): void
{
    $compteSource = new Compte();
    $compteSource->setNumeroCompte('UNI-SRC-001');

    $this->transaction->setCompteSource($compteSource);
    $this->transaction->setType(TransactionType::DEPOT);
    $this->transaction->setMontant('3000.00');

    $this->assertSame(TransactionType::DEPOT, $this->transaction->getType());

```

```

        $this->assertEquals('3000.00', $this->transaction->getMontant());
    }

    /**
     * Regle metier 5 : Les trois types de transaction existent
     */
    public function testAllTransactionTypes(): void
    {
        $this->transaction->setType(TransactionType::VIREMENT);
        $this->assertSame(TransactionType::VIREMENT, $this->transaction->getType());

        $this->transaction->setType(TransactionType::RETRAIT);
        $this->assertSame(TransactionType::RETRAIT, $this->transaction->getType());

        $this->transaction->setType(TransactionType::DEPOT);
        $this->assertSame(TransactionType::DEPOT, $this->transaction->getType());
    }

    /**
     * Regle metier 6 : La description est optionnelle
     */
    public function testDescriptionIsOptional(): void
    {
        $this->assertNull($this->transaction->getDescription());

        $this->transaction->setDescription('Palement facture');
        $this->assertEquals('Palement facture', $this->transaction->getDescription());
    }

    /**
     * Regle metier 7 : Le compte destination est nullable (pour retrait et depot)
     */
    public function testCompteDestinationNullable(): void
    {
        $this->assertNull($this->transaction->getCompteDestination());
    }

    /**
     * Regle metier 8 : Simulation d'un virement avec mise a jour des soldes
     */

```

```

public function testVirementUpdatesSoldes(): void
{
    $compteSource = new Compte();
    $compteSource->setSolde('5000.00');

    $compteDest = new Compte();
    $compteDest->setSolde('1000.00');

    $montant = 2000.00;

    $nouveauSoldeSource = number_format((float)$compteSource->getSolde() - $montant, 2, '.', '');
    $nouveauSoldeDest = number_format((float)$compteDest->getSolde() + $montant, 2, '.', '');

    $compteSource->setSolde($nouveauSoldeSource);
    $compteDest->setSolde($nouveauSoldeDest);

    $this->assertEquals('3000.00', $compteSource->getSolde());
    $this->assertEquals('3000.00', $compteDest->getSolde());
}
}

```

4 - Resume

Fichier de Test	Tests	Assertions	Resultat
CompteTest.php	10	16	OK
TransactionBancaireTest.php	8	19	OK
TOTAL	18	35	100% PASS

PHPStan : 0 erreurs (niveau 5) sur 6 fichiers analyses

PHPUnit : 18 tests, 35 assertions - 100% de reussite